XStore

Xcelerated performance in time-series key/value STORagE systems

Lan Nguyen & Dr. Ioan Raicu
Data-Intensive Distributed Systems Laboratory

ILLINOIS TECH

DataSys
Data-Intensive Distributed Systems Laboratory

Table of Contents

Introductions

Research Problems

XStore

Evaluations

Future Work

Q/A

Research Domain

Data Storage:

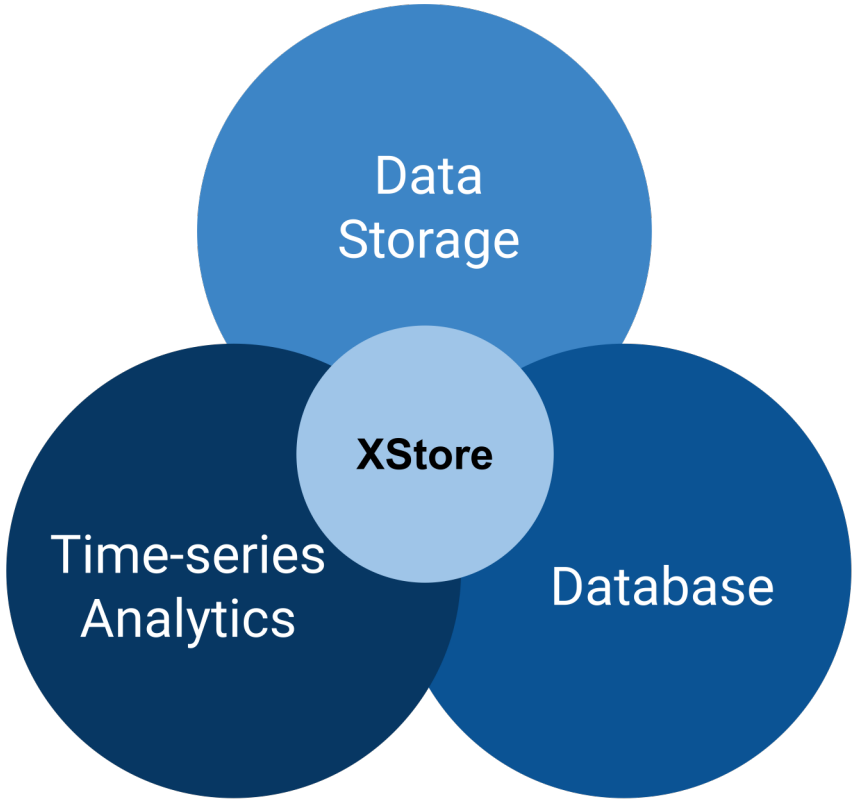
- Stores data in a semi-organized on various medium ranging from RAM, HDD and SSD
- Applications: Network File-Systems (NFS), Parallel File-Systems (PFS) and Distributed File-Systems (DFS)

Database:

- Data storage management system designed to hold and maintain data in a pre-defined structure
- Applications: TimescaleDB, InfluxDB and MongoDB

Time-Series Analytics:

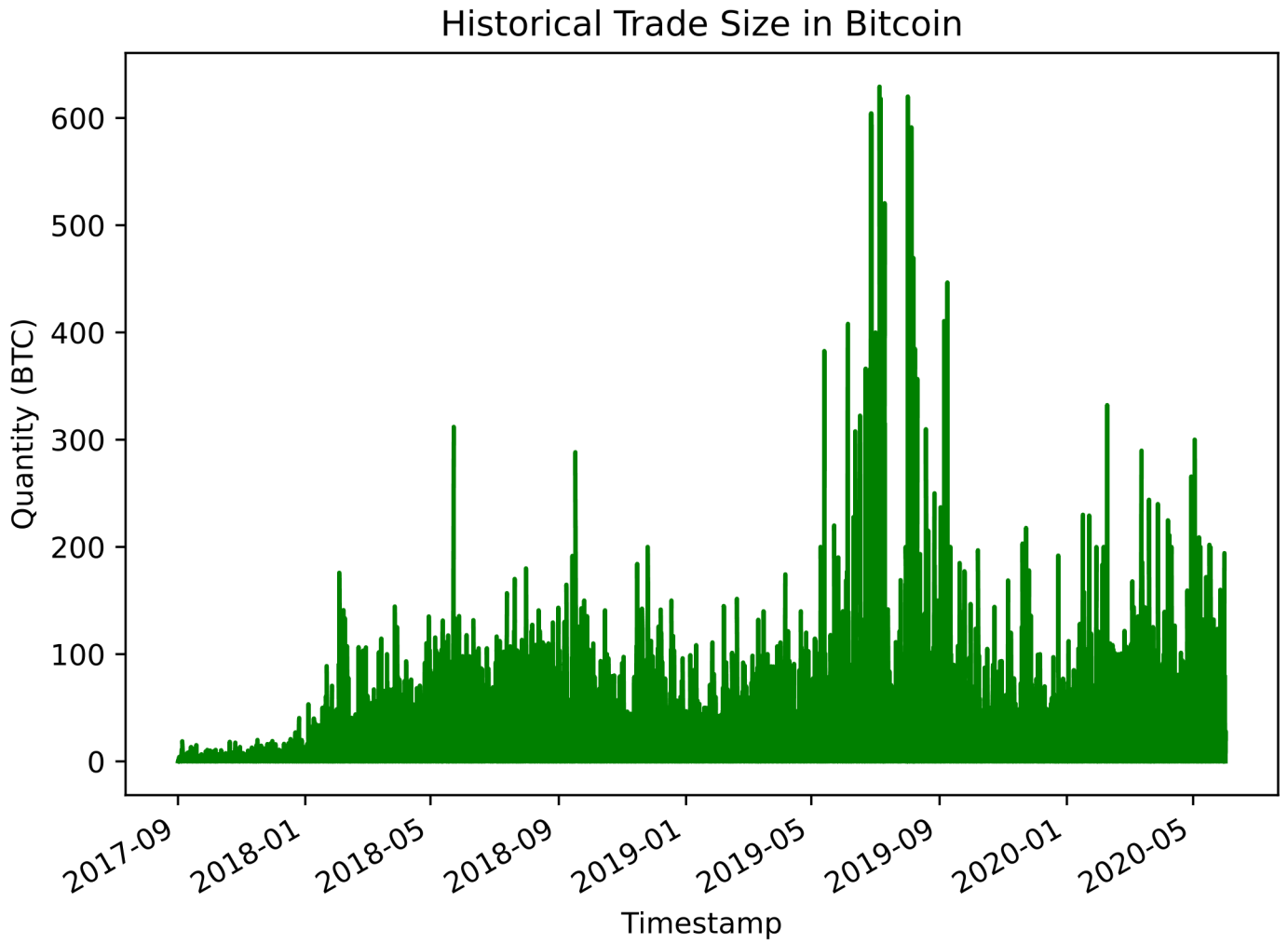
- Extracting useful insights from a successive sequence of data points organized by time and measured over a time interval
- Applications: Financial engineering, scientific simulation, health analytics



Example of a Time-series table

Timestamp	Price ($)	Qty (BTC)	Trade ID	First tradeID	Last tradeID	Buyer marker	...
2017-09-01 00:01:00.493	4689.89	0.053417	61458	69180	69180	True	...
2017-09-01 00:01:00.530	4689.89	0.064732	61459	69181	69182	True	...
2017-09-01 00:01:00.573	4689.90	0.147065	61460	69183	69183	True	...
2017-09-01 00:01:18.120	4689.90	0.042047	61461	69184	69184	True	...
2017-09-01 00:01:18.210	4689.91	0.115178	61462	69185	69185	True	...
...	...	...	...	...	...	...	...

Example of a Time-series chart



Time-Series Data Overview

1 **US Stocks**

Millisecond-granular price data of ≥ 4,500 tickers from 2010*:

- ≥ 200 (TB)
- 90b rows
- 450b metrics

2 **Visa**

Visa can process up to 24,000 transactions per second+. 10 years data:

- 7.5 trillion rows

How can we retrieve time-series data regardless of its scale?

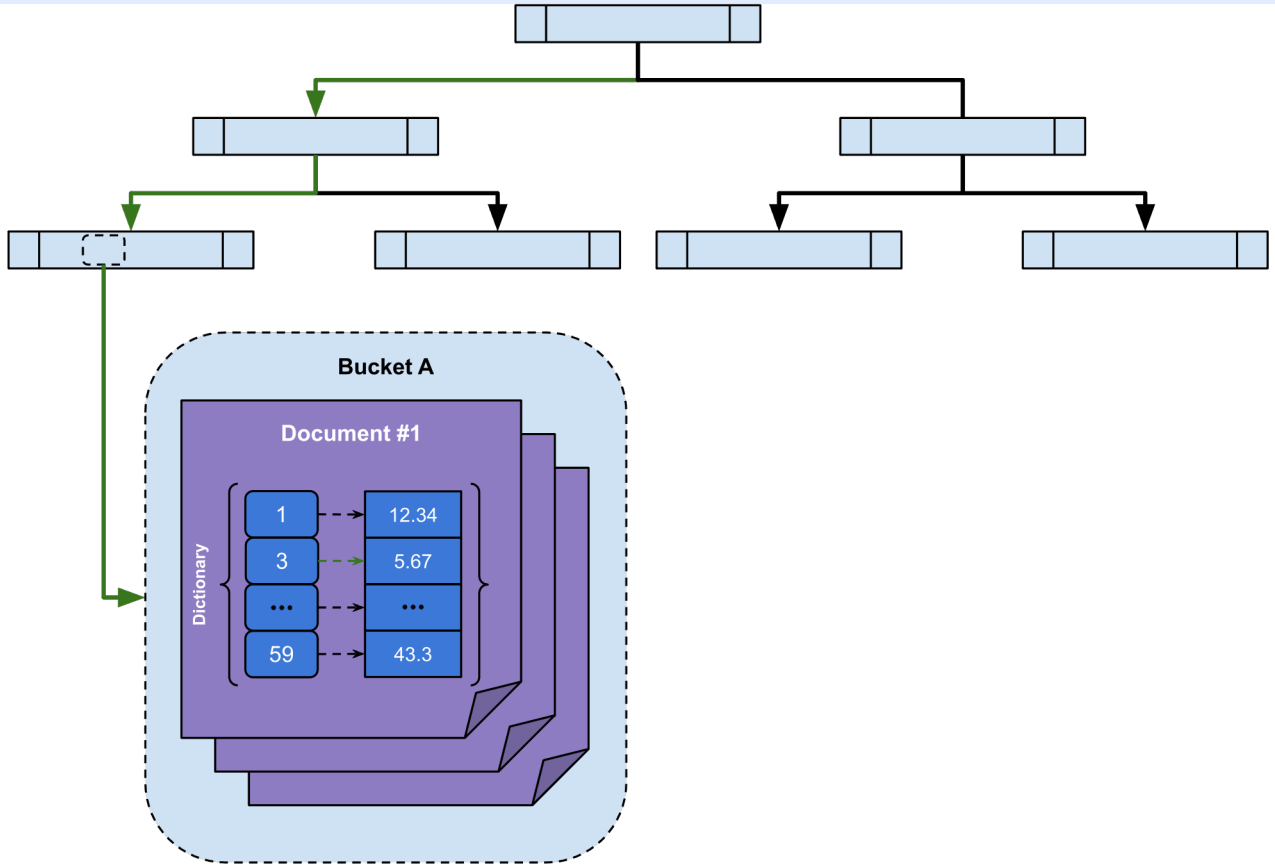
*https://firstratedata.com/tick-data
+https://shorturl.at/vEJS3

Research Problems

1	Chrono Structure	Current's cutting-edge solutions lack the performance acceleration through the adoption of structured time-series data
2	Reliance on Tree	Maintenance of *tree* structures require a sizable amount of memory to maintain an $O(log_n)$ complexity
3	Scalability	Current solutions lack support for fine-grain time-series data while scability is a concern due to its reliance on Tree structures
4	Index	Databases accelerate its performance through indexes. This incurs a heavy tax on CPU and memory while not reaching *O(1)* performance



Related Work (MongoDB)



Granularity	Bucket Time Span	No. Documents	Rows per Documents
Seconds	1 Hour	18	200
Minutes	24 Hours	6	240
Hours	30 Days	2	360

Introducing XStore

XStore is a high-performance time-series key-value store designed to address performance gap in data retrieval in an ever-growing number of time-series. Equipped with techniques that leverage an order of magnitude of time to deliver an extreme performance

1	Performance	Guarantees *O(1)* complexity for CRUD* operations
2	System's Resource Efficiency	Does not maintain index or hash-map, reduces processor and memory costs
3	Scalable	Maintains *O(1)* performance regardless of granularity and size of time-series data
4	Persistent	Employs "Store nothing in memory, always on disk" policy







ILLINOIS TECH DataSys

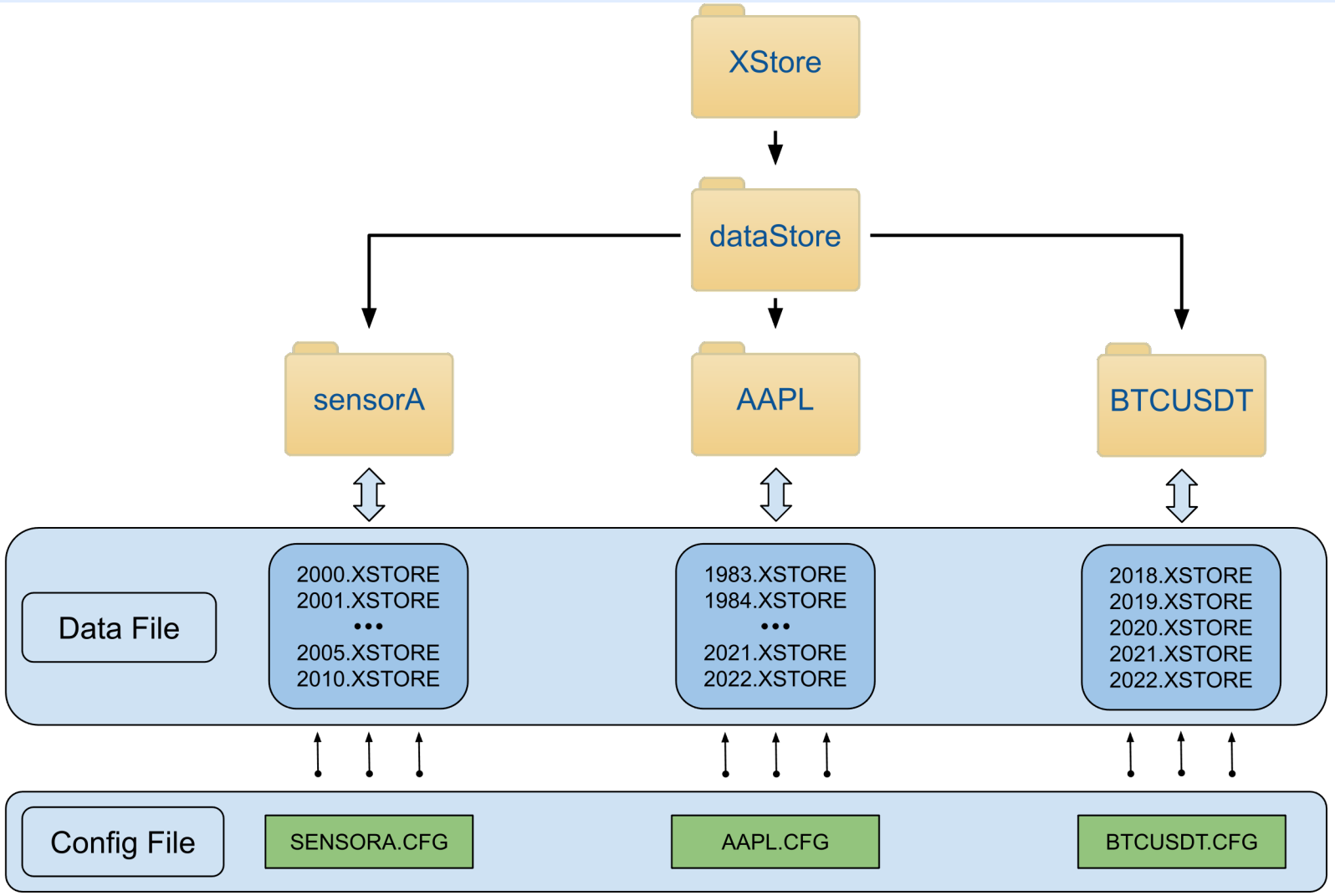
*CRUD – Create, Read, Update and Delete

Order Magnitude of Time

1-Year worth of data

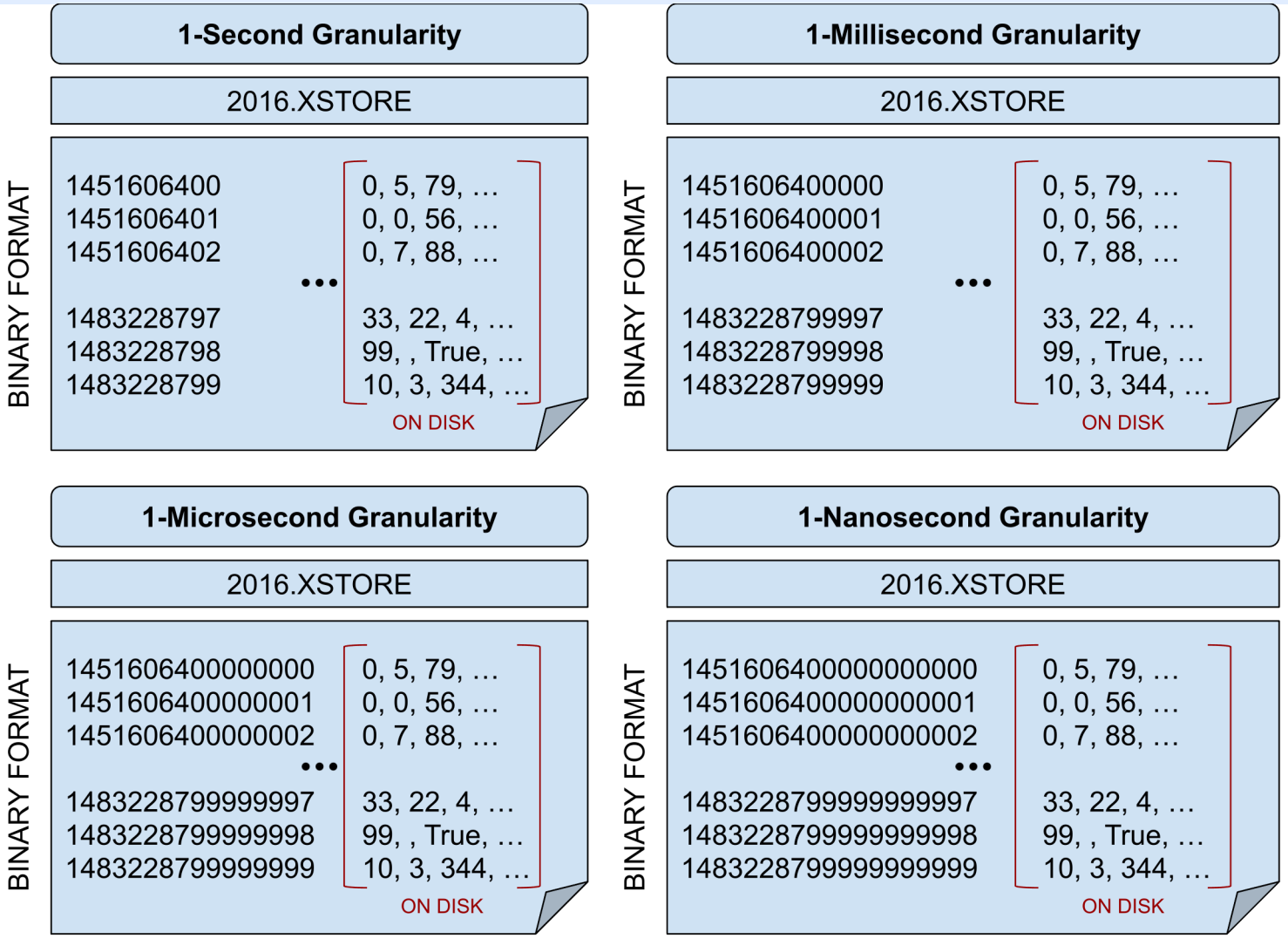
Granularity	No. of Entries
1-Month	12
1-Week	52
1-Day	365
1-Hour	8,760
1-Minute	525,960
1-Second	31,536,000
1-Millisecond	31,536,000,000
1-Microsecond	31,536,000,000,000
1-Nanosecond	31,536,000,000,000,000

File Organization



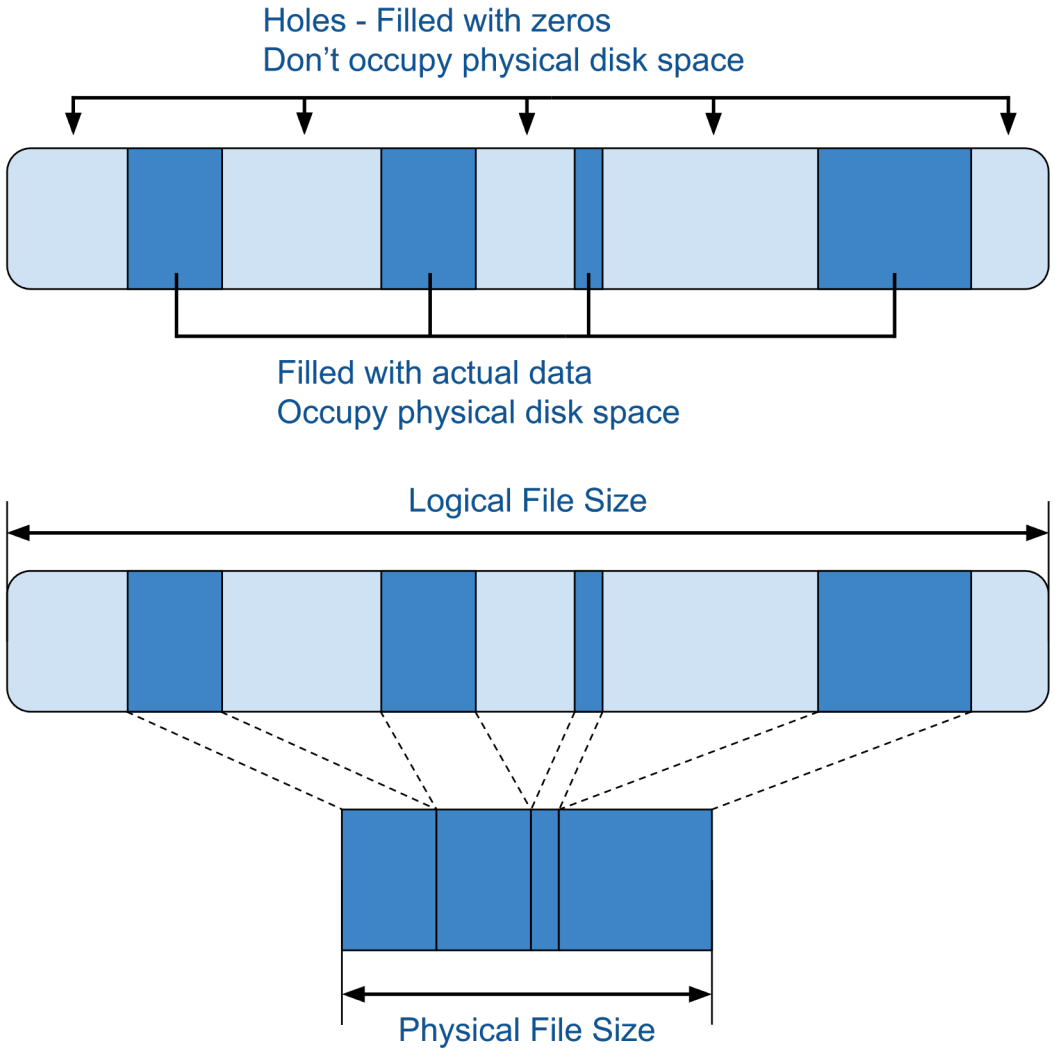
Note: Each DB always associated with a single .CFG file

Data Organization



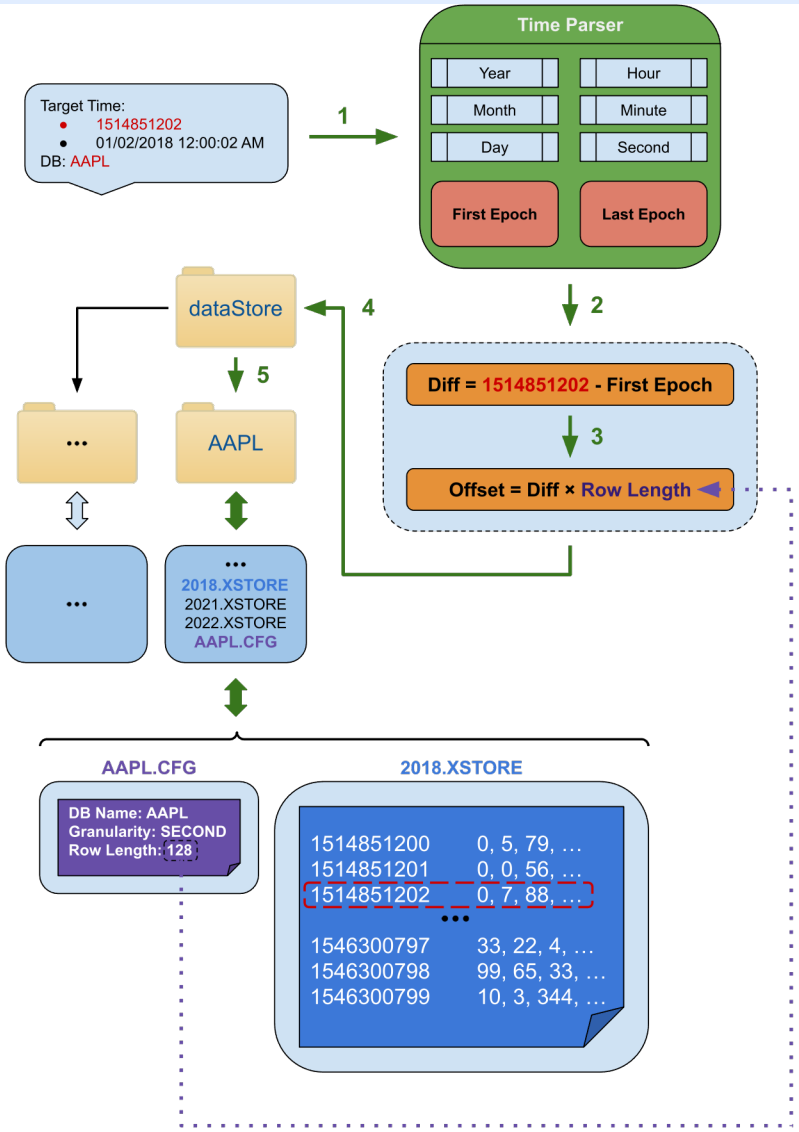
Note: Epoch time is computed at run-time based on file name

Sparse File

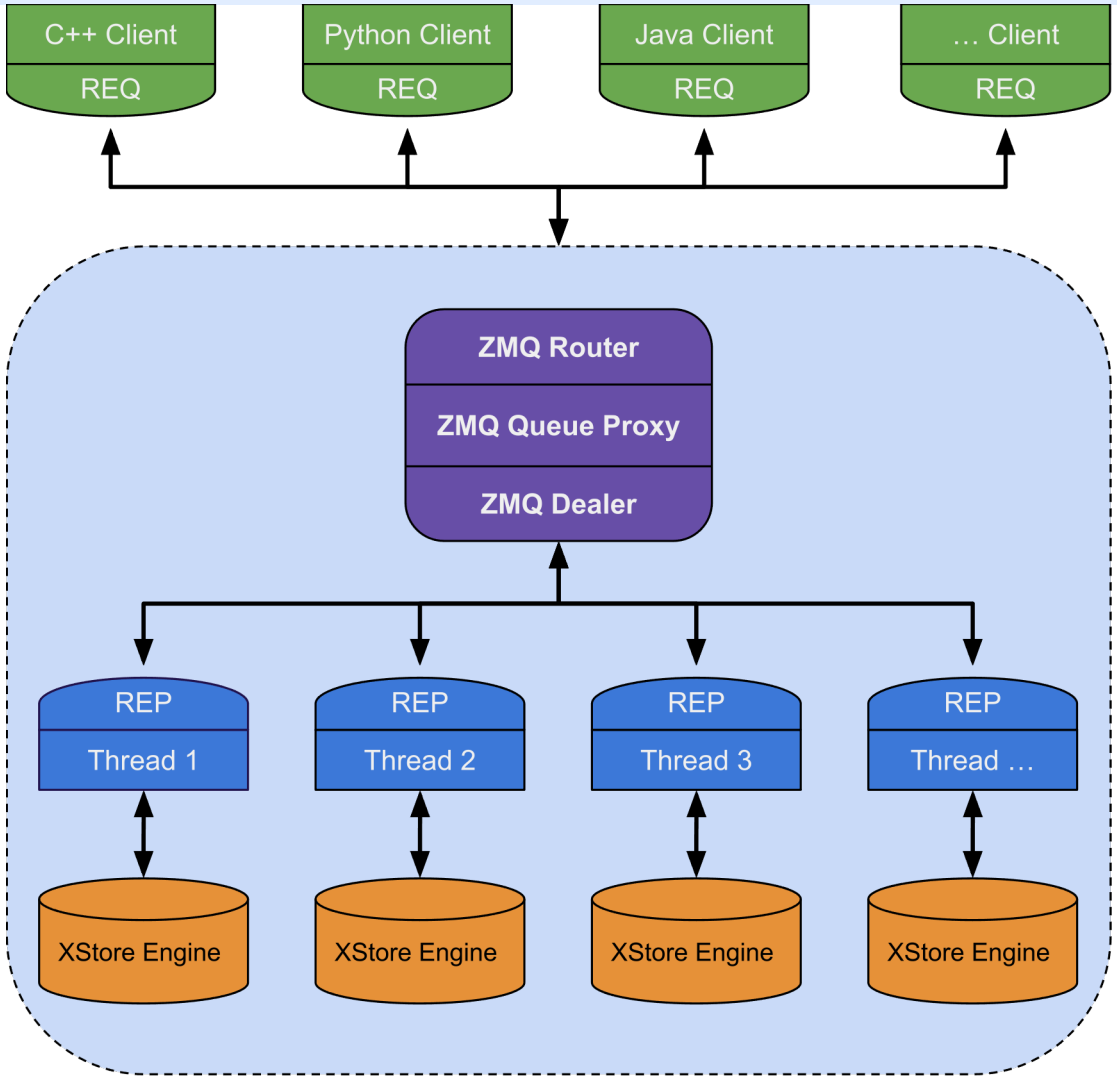


Source: https://en.wikipedia.org/wiki/Sparse_file

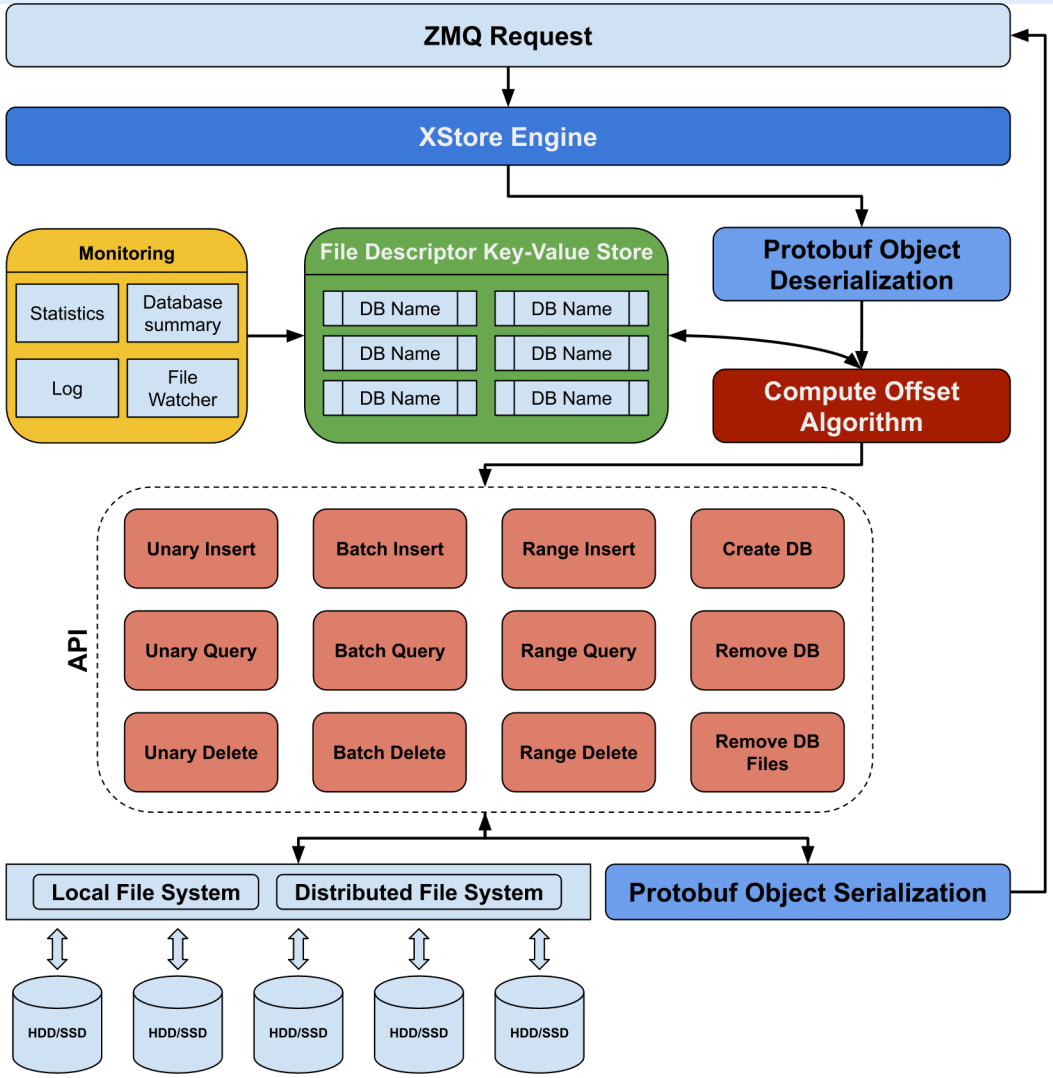
Offset Algorithm



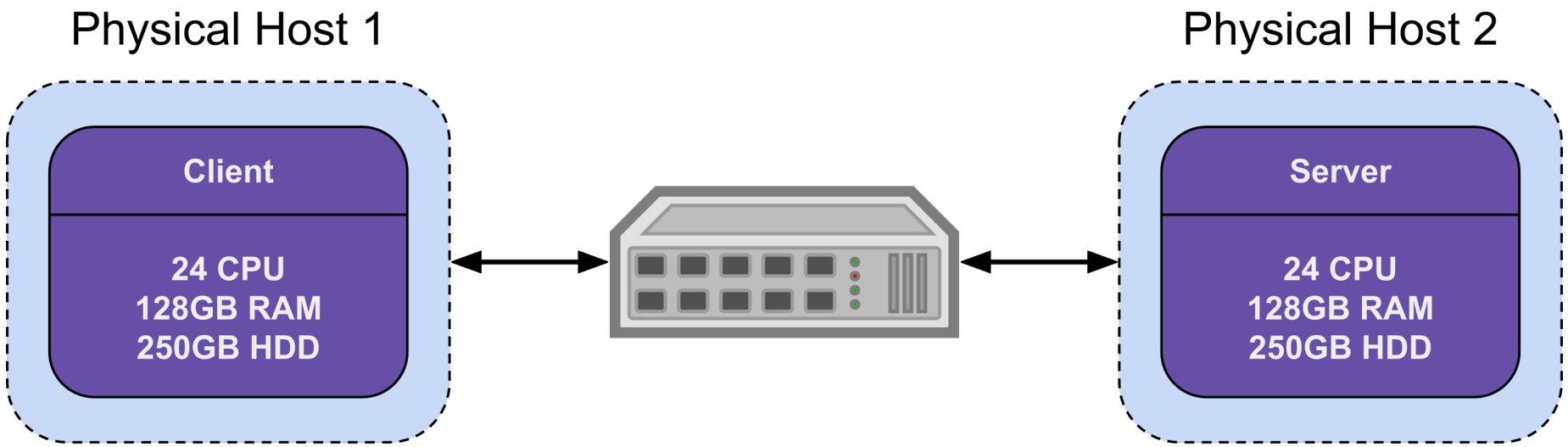
Outer Layer – Networking



Inner Layer – XStore Engine



Benchmark Configurations



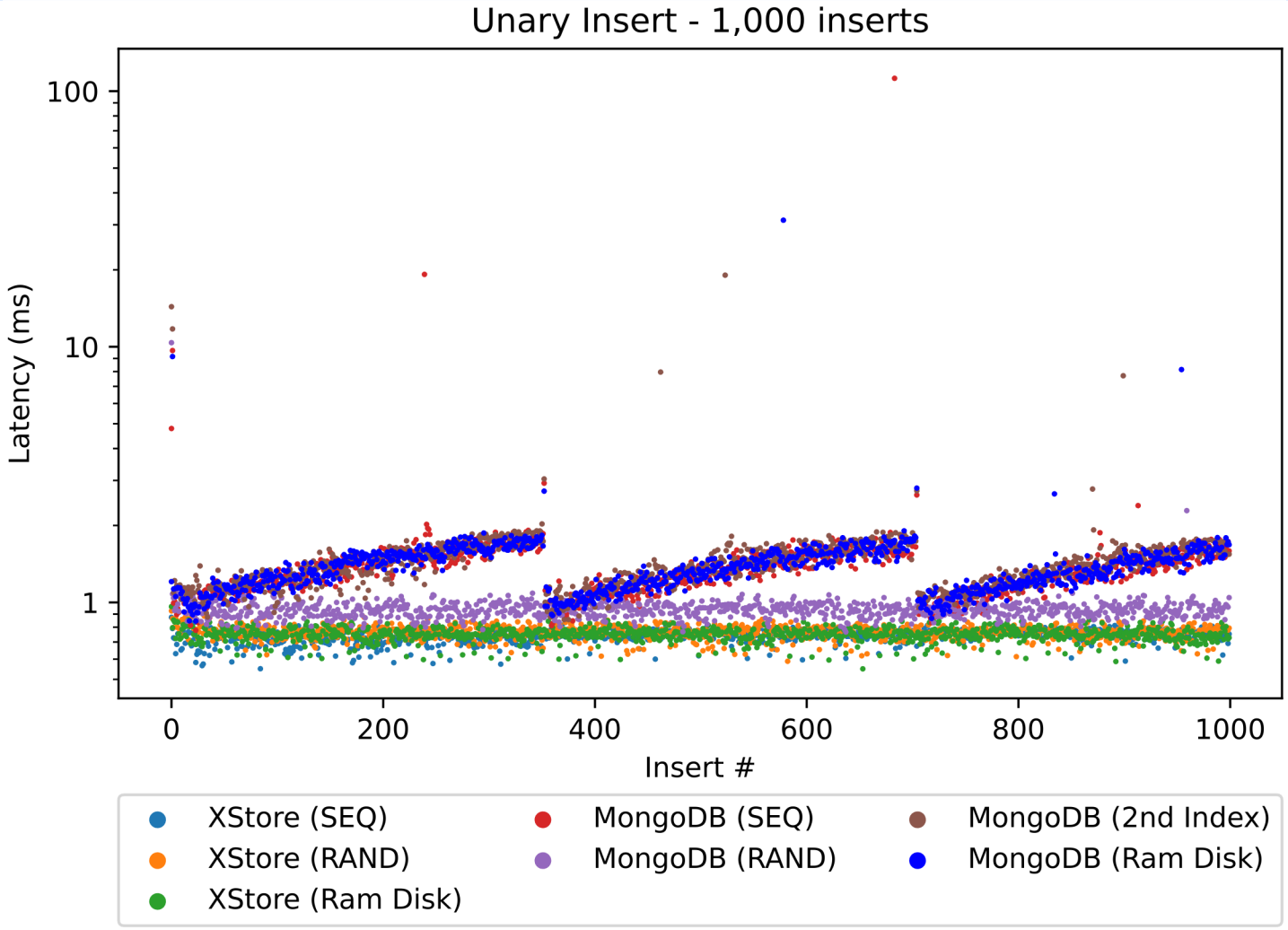
Physical Host OS: Linux Ubuntu 20.04
Client Code is in *Python*
Filesystem: EXT4
Storage: Seagate ST9250610NS SATA 7200 RPM HDD (64MB cache)
Seek Time, Avg. Read/Write = 8.5/9.5 (ms)
Avg. Latency = 4.16 (ms)
Network: 10 Gigabit Ethernet

Synthetic Data

Timestamp	firstCol	...	eighthCol
946684800	396645bc59122e7b88d9adb9d9e1c218	...	d55af0d89abb2c79f91a58e2c5898de0
946684801	d42dfbbceb10676fab12306ed5f3d300	...	e666bc094a7c52320cca96da2d7b67ae
...	...	...	...
1609459198	a6df9f30ca626cef52d2932fa3e5119d	...	4b2e36596cd2dbc11a391297855db2fa
1609459199	6a49b9b7a3fd25b428f5439fc98e561e	...	b6897b2c72c52e348f0236cac60c3165

Synthetic data ranges over 20 years from 01-01-2000 to 12-31-2020
Each column is a hash value of timestamp. EX: Value of firstCol = MD5Sum(Timestamp + "-1")
Granularity: SECONDS

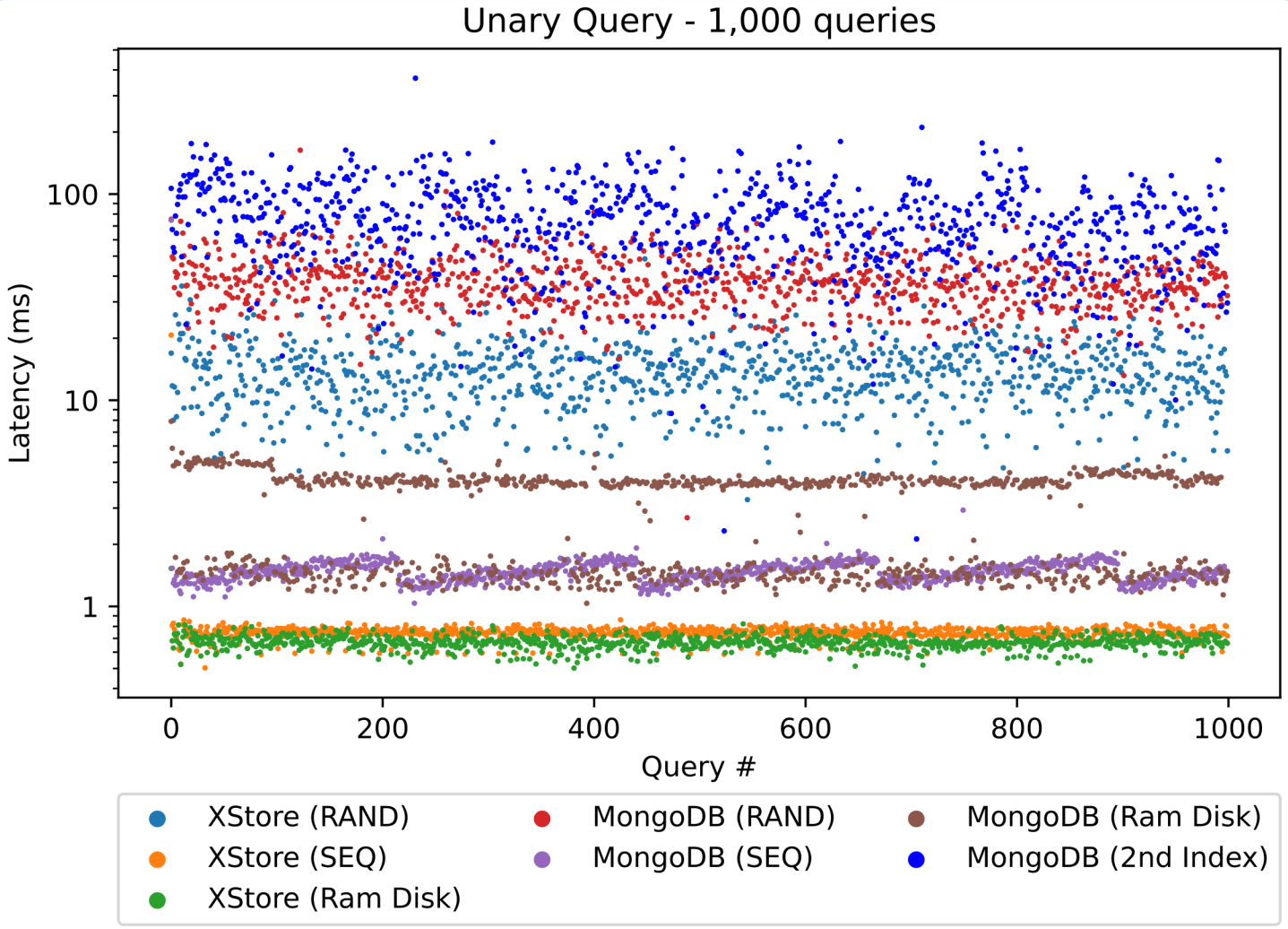
Benchmark – Unary Insert



	Min	Max	Mean	Std Dev	Avg. Throughput
XStore (SEQ)	0.55	0.87	0.74	0.04	1,352.65
XStore (RAND)	0.59	0.93	0.76	0.05	1,307.44
XStore (Ram Disk)	0.55	0.96	0.75	0.05	1,336.26
MongoDB (SEQ)	0.78	1.95	1.34	0.24	744.98
MongoDB (RAND)	0.77	1.10	0.93	0.06	1,070.60
MongoDB (2nd Index)	0.86	19.06	1.45	0.86	688.02
MongoDB (Ram Disk)	0.85	31.27	1.41	1.03	711.08

Note: Ram Disk benchmarks are run with Sequential Workload

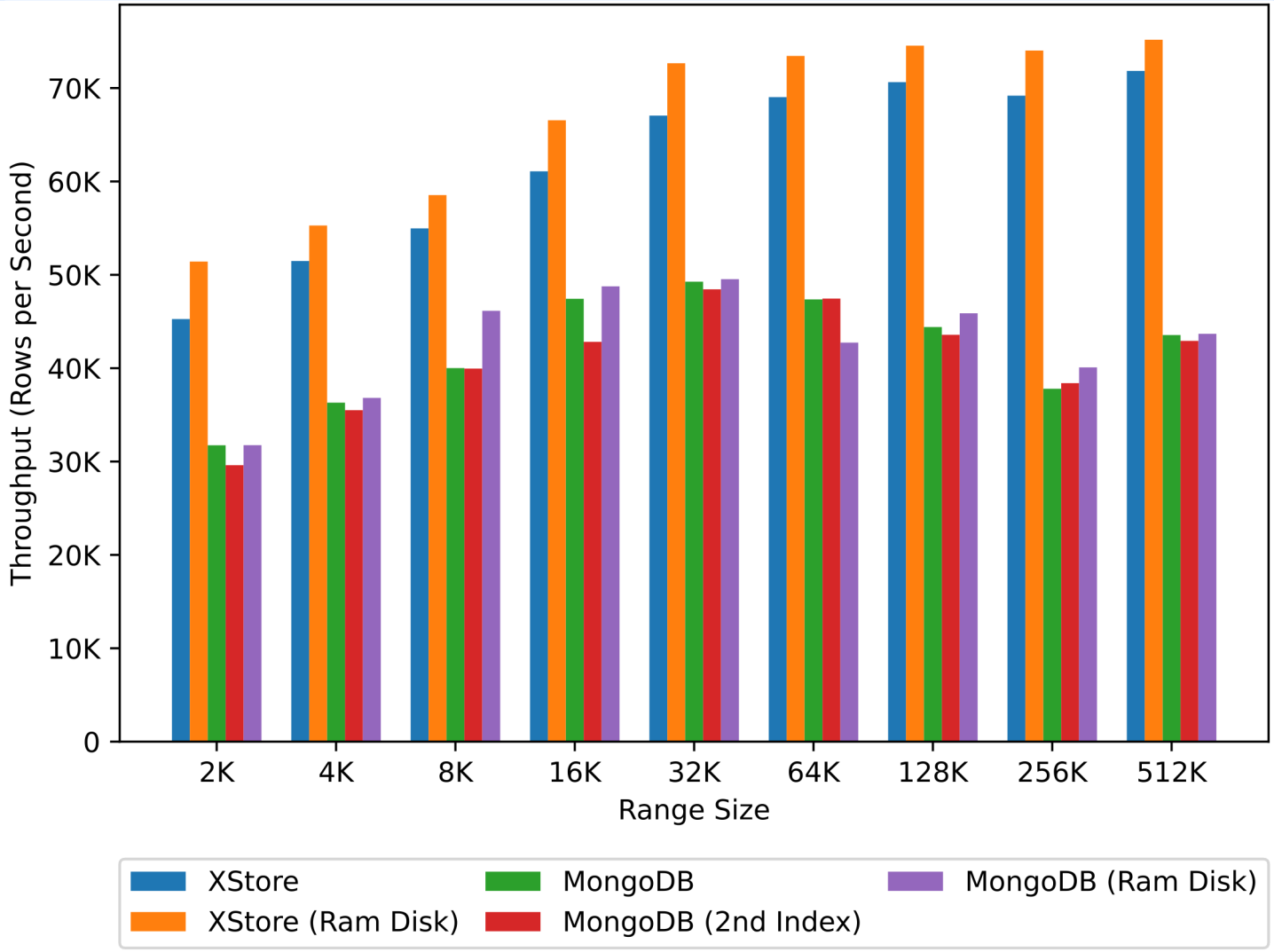
Benchmark – Unary Query



	Min	Max	Mean	Std Dev	Avg. Throughput
XStore (SEQ)	0.50	20.70	0.76	0.63	1,309.49
XStore (RAND)	3.29	77.52	13.89	5.57	71.97
XStore (Ram Disk)	0.50	1.53	0.67	0.06	1,494.94
MongoDB (SEQ)	1.04	74.71	1.55	2.32	644.86
MongoDB (RAND)	2.69	163.24	37.91	11.78	26.38
MongoDB (2nd Index)	2.12	364.71	75.27	34.36	13.29
MongoDB (Ram Disk)	1.04	7.90	2.93	1.39	340.97

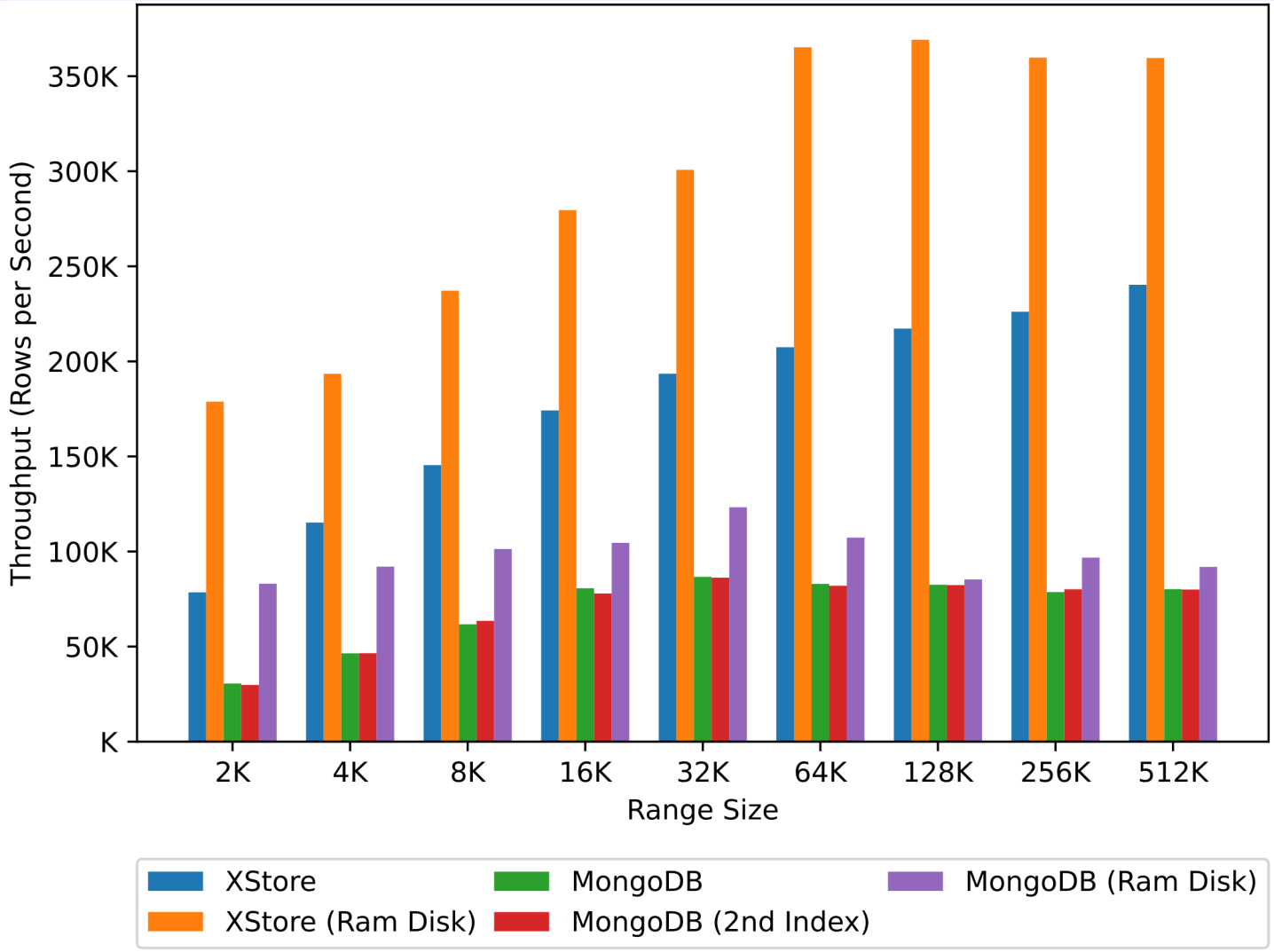
Note: Ram Disk benchmarks are run with Random Workload

Benchmark – Range Insert (Average Throughput)



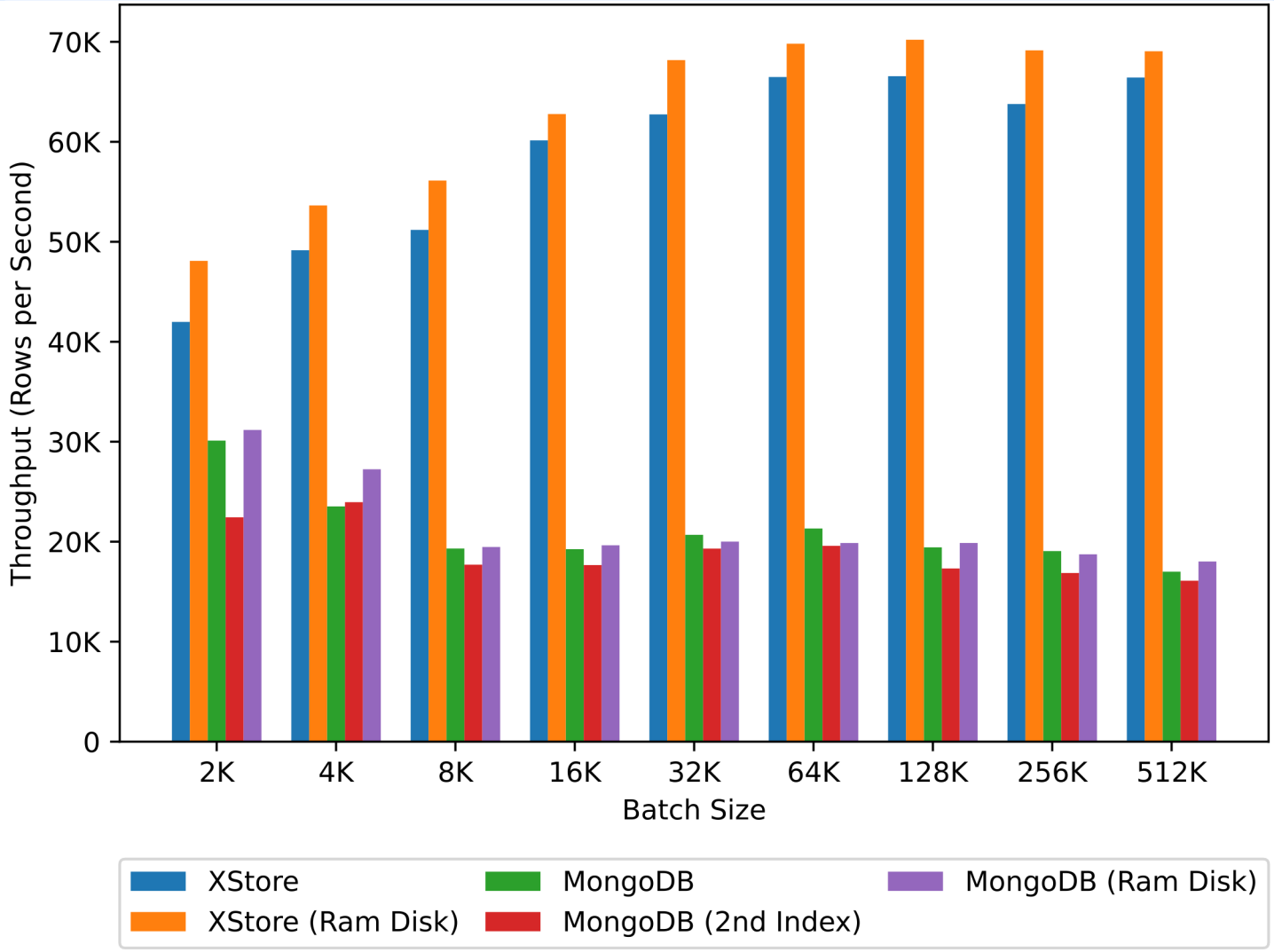
Note: 10 iterations for each range size

Benchmark – Range Query (Average Throughput)



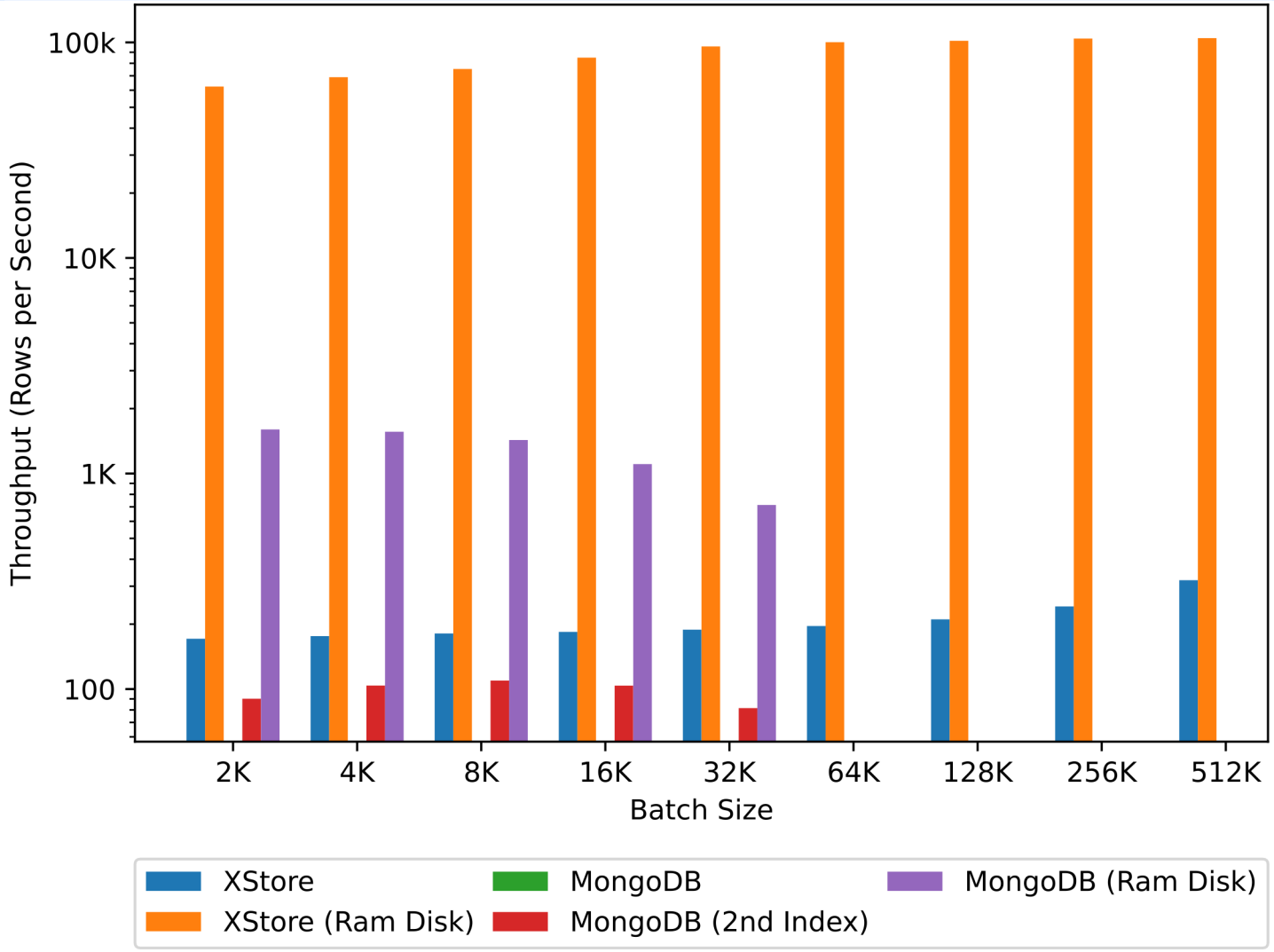
Note: 1,000 iterations for each range size

Benchmark – Batch Insert (Average Throughput)



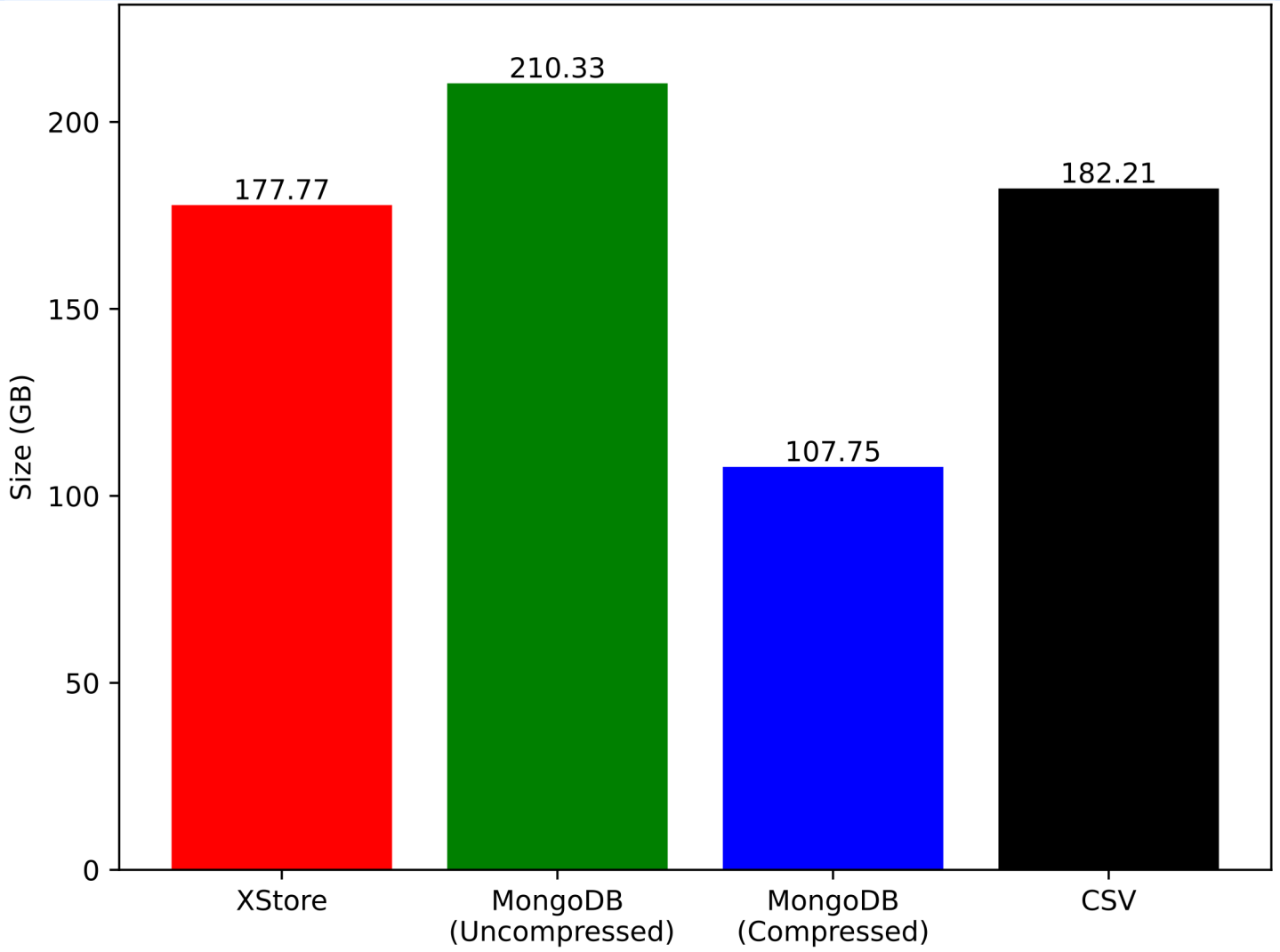
Note: 10 iterations for each range size

Benchmark – Batch Query (Average Throughput)



Note: 10 iterations for each range size

Benchmark – Storage Footprint



Benchmark – Summary

1	Performance	Empirical data shows a consistent outperformance (2x to 4x) by utilizing time structure (XStore) over bucket-approach (MongoDB) across all workloads
2	Scalability	Fine-grained data poses a big threat to bucket-approach (MongoDB)
3	Random Read	Batch Query benchmark emphasizes drawback of bucket-approach
4	Storage Size	MongoDB consumes 40% less storage space than XStore due to compression

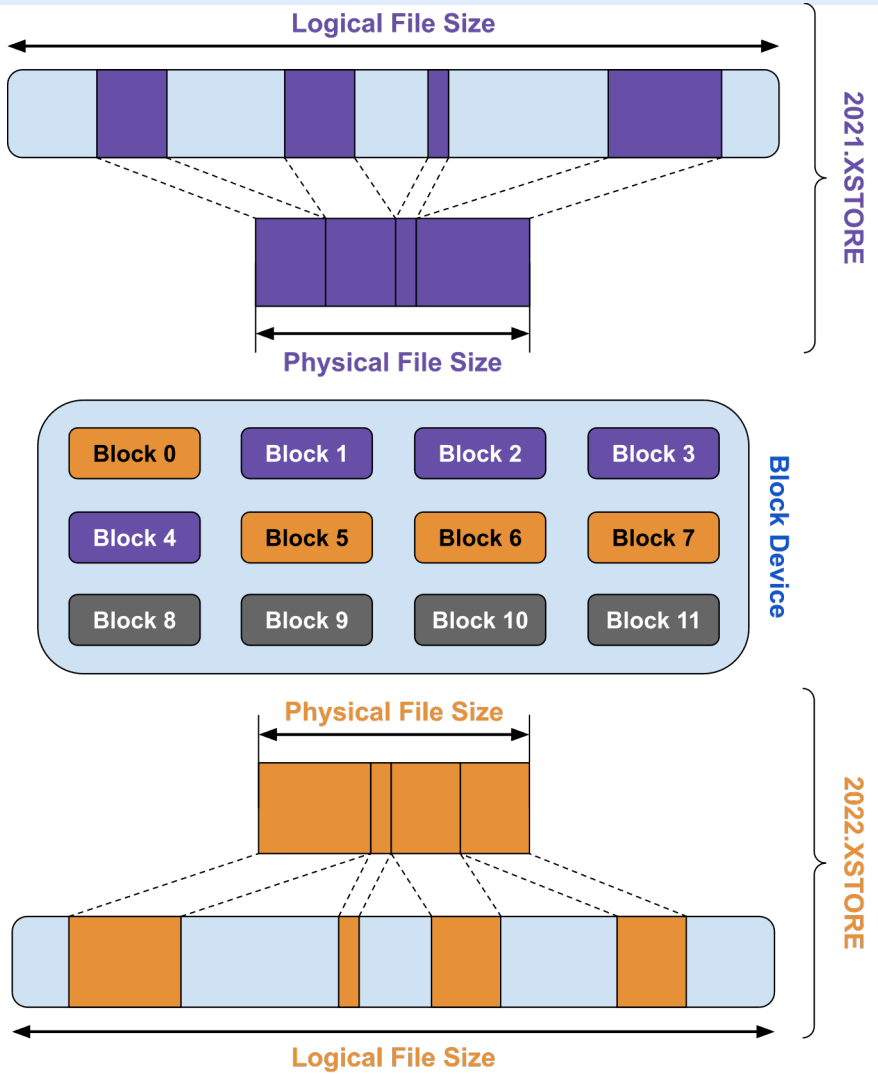
Future Work

1	Real Use-Case	Perform benchmark on a real use-case such as conducting a backtest and parameter optimization in financial engineering
2	Benchmark	More comprehensive benchmark can be but not limited to benchmark on physical machines, data stored on DFS, additional TSDB, etc.
3	Performance	Revise existing codebase & Enhance its performance
4	Search	Integrate XSearch/SCANNS* into XStore
5	Distributed	Conduct analyses on Distributed Filesystems (Lustre, Cephfs)
6	Publication	Aim for publication to IEEE/ACM SuperComputing/SC

*Alexandru Iulian Orhean, Anna Giannakou, Lavanya Ramakrishnan, Kyle Chard, and Ioan Raicu. Scanns: Towards scalable and concurrent data indexing and searching in high-end computing system. In 2022 22nd IEEE International Symposium on Cluster, Cloud and Internet Computing (CCGrid), pages 51–60. IEEE, 2022.

ILLINOIS
TECH

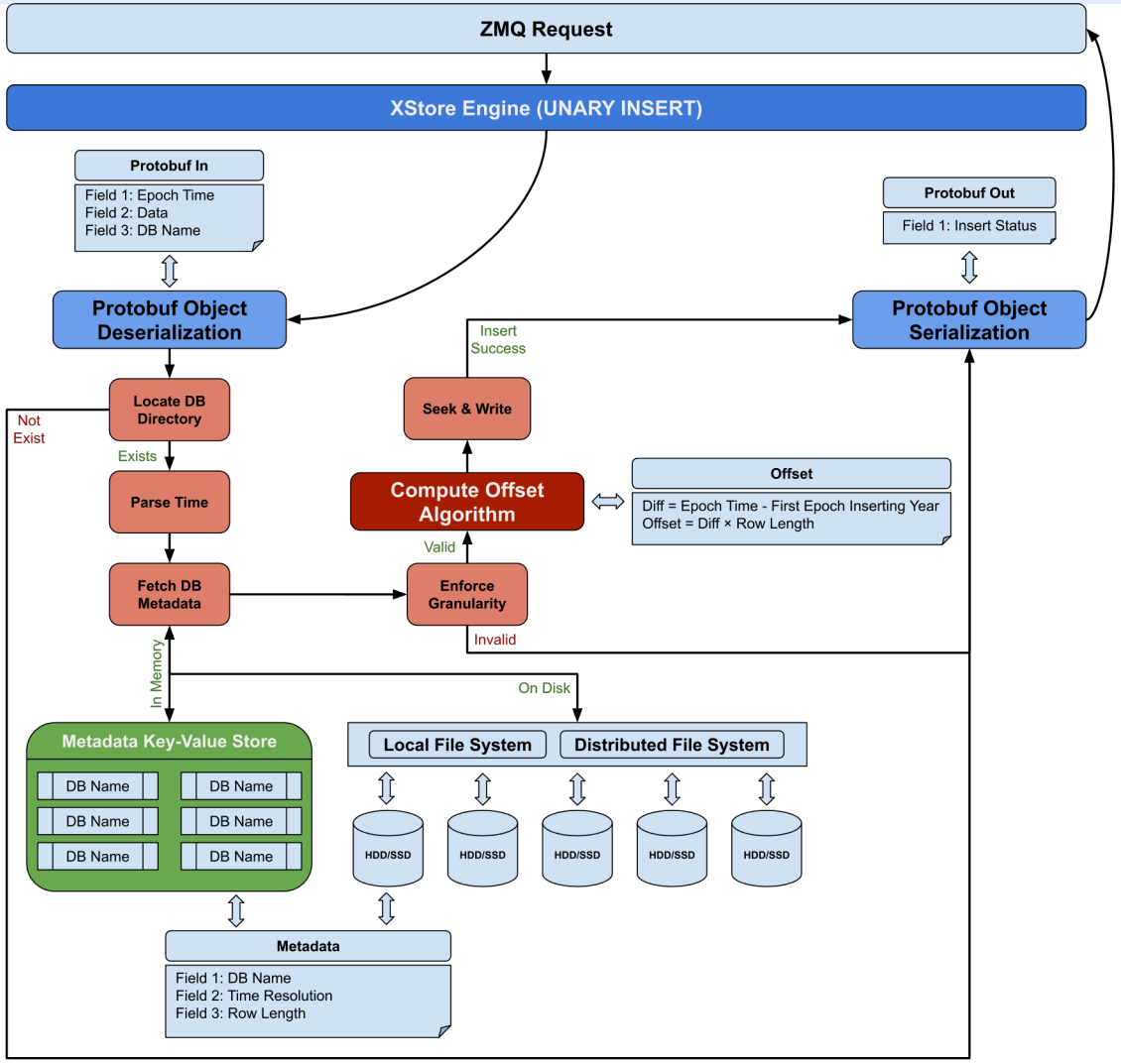
Fragmentation



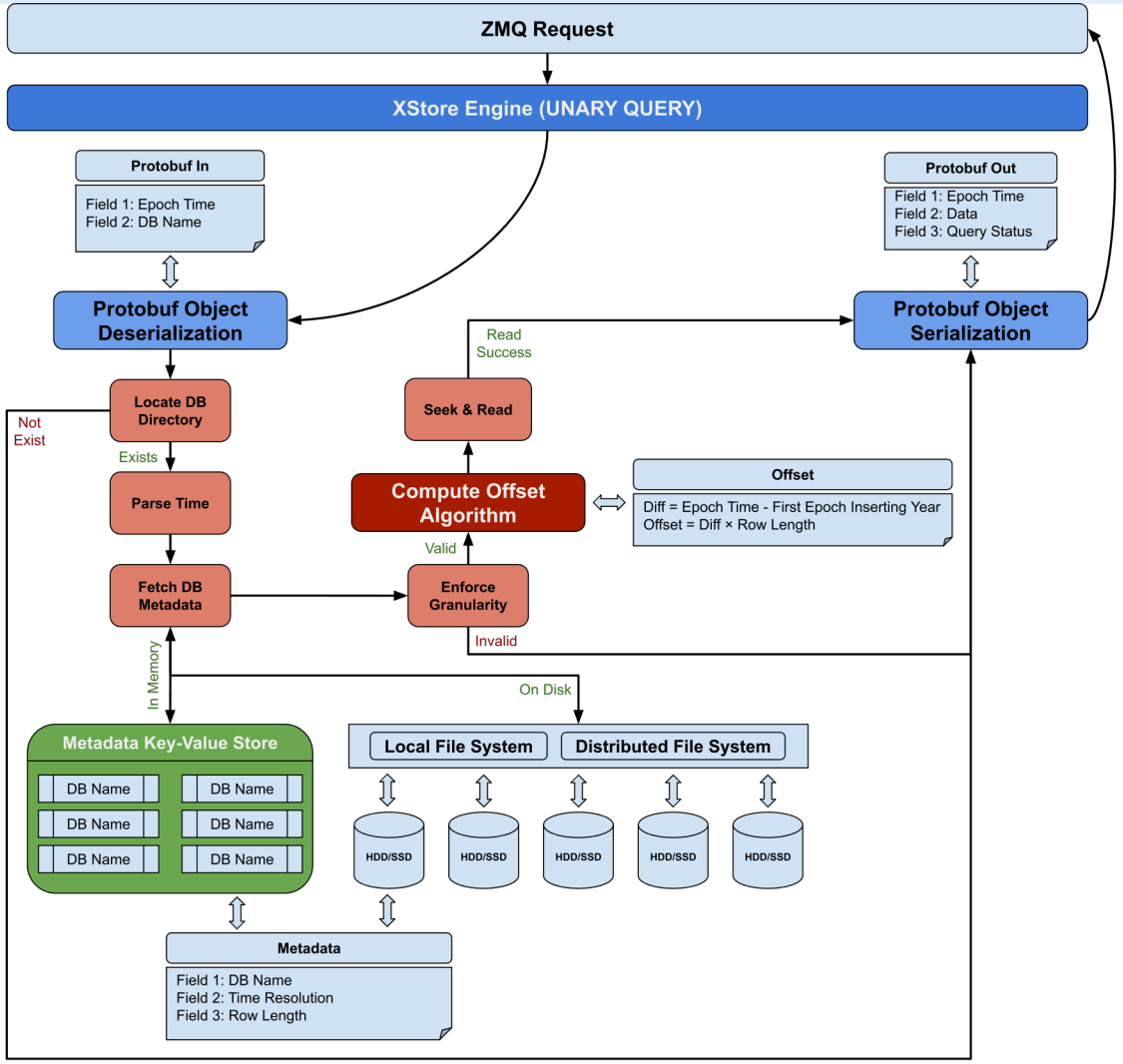
Fragmentation

```
cc@tacc-1:~/XStore_Server/dataStore/SYNTHDATA$ filefrag -v 2000.XSTORE
Filesystem type is: ef53
File size of 2000.XSTORE is 9107251200 (2223450 blocks of 4096 bytes)
 ext:     logical_offset:        physical_offset: length:   expected: flags:
   0:        0..   28671:    3117056..   3145727:  28672:
   1:    28672..   61439:    3178496..   3211263:  32768:    3145728:
   2:    61440..   94207:    3211264..   3244031:  32768:
   3:    94208..  126975:    3244032..   3276799:  32768:
   4:   126976..  159743:    3276800..   3309567:  32768:
   5:   159744..  192511:    3309568..   3342335:  32768:
   6:   192512..  225279:    3342336..   3375103:  32768:
   7:   225280..  258047:    3375104..   3407871:  32768:
   8:   258048..  290815:    3407872..   3440639:  32768:
   9:   290816..  323583:    3440640..   3473407:  32768:
  10:   323584..  356351:    3473408..   3506175:  32768:
  11:   356352..  389119:    3506176..   3538943:  32768:
  12:   389120..  421887:    3538944..   3571711:  32768:
  13:   421888..  454655:    3571712..   3604479:  32768:
  14:   454656..  487423:    3604480..   3637247:  32768:
  15:   487424..  520191:    3637248..   3670015:  32768:
  16:   520192..  552959:    3702784..   3735551:  32768:    3670016:
  17:   552960..  585727:    3735552..   3768319:  32768:
  18:   585728..  618495:    3768320..   3801087:  32768:
  19:   618496..  651263:    3801088..   3833855:  32768:
  20:   651264..  684031:    3833856..   3866623:  32768:
  21:   684032..  716799:    3866624..   3899391:  32768:
  22:   716800..  749567:    3899392..   3932159:  32768:
  23:   749568..  782335:    3932160..   3964927:  32768:
  24:   782336..  815103:    3964928..   3997695:  32768:
  25:   815104..  847871:    3997696..   4030463:  32768:
  26:   847872..  880639:    4030464..   4063231:  32768:
  27:   880640..  913407:    4063232..   4095999:  32768:
  28:   913408..  946175:    4098048..   4130815:  32768:    4096000:
  29:   946176..  978943:    4130816..   4163583:  32768:
  30:   978944.. 1009663:    4163584..   4194303:  30720:
  31:  1009664.. 1042431:    4227072..   4259839:  32768:    4194304:
```

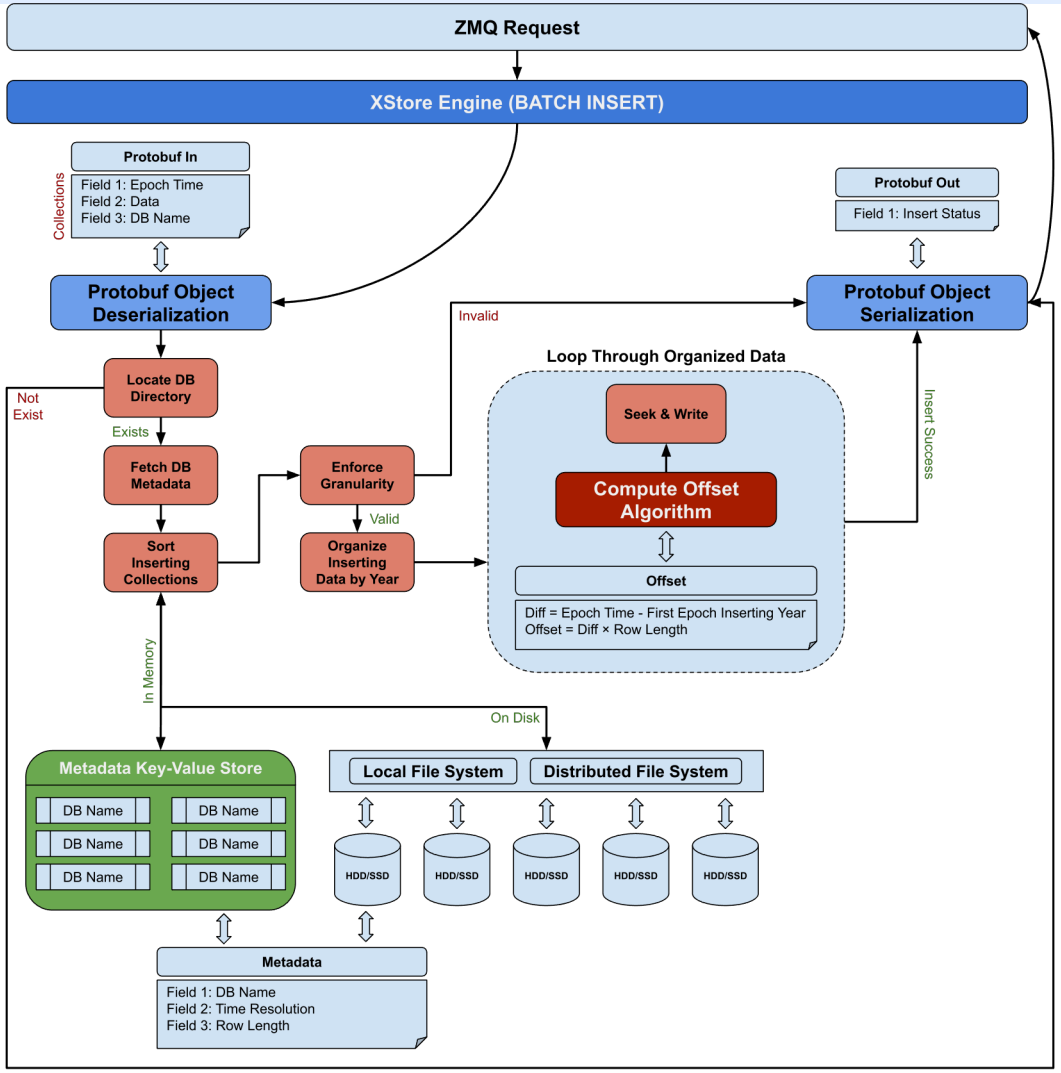
XStore Engine – Unary Insert



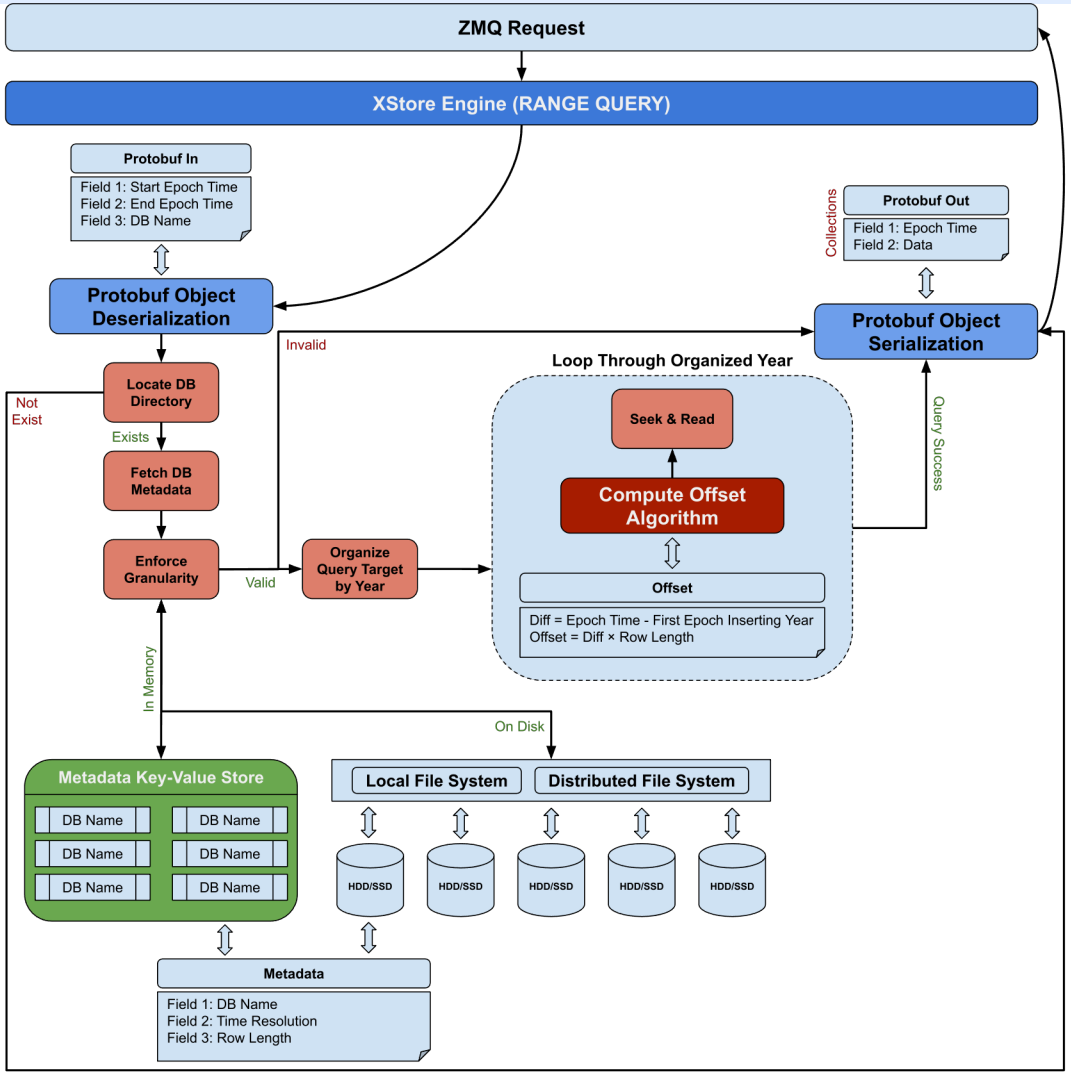
XStore Engine – Unary Query



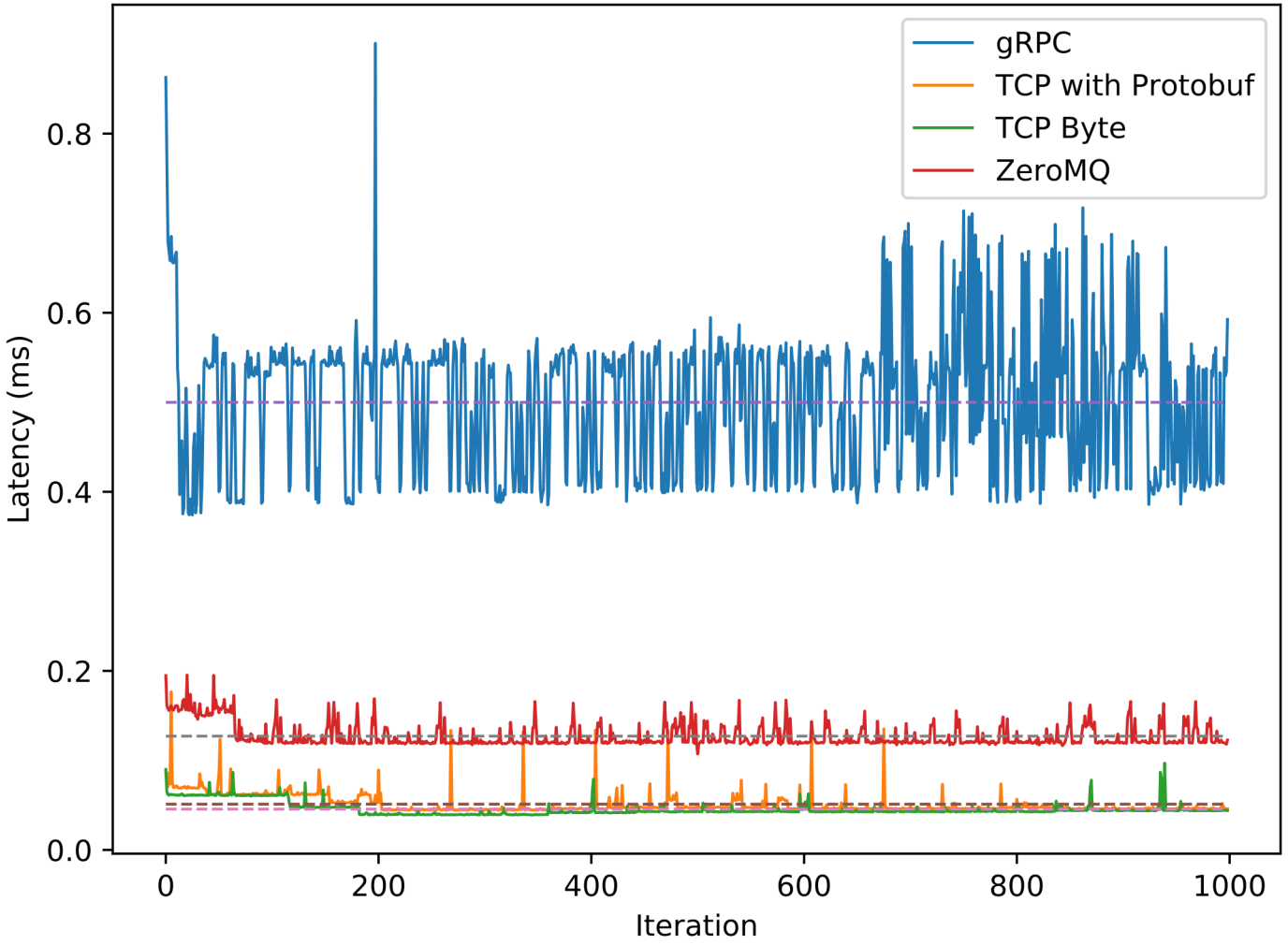
XStore Engine – Batch Insert



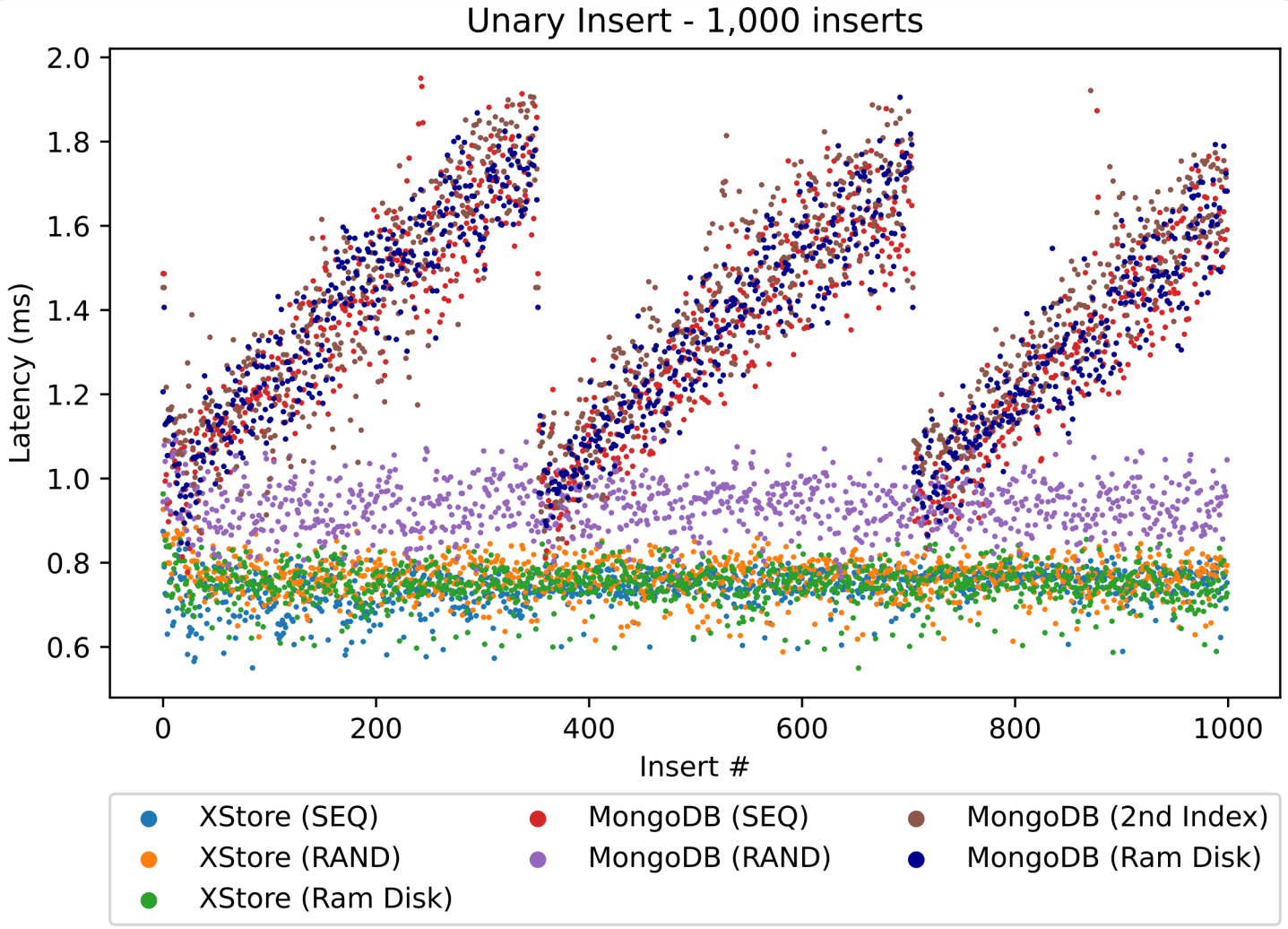
XStore Engine – Range Query



Microbenchmark – Networking



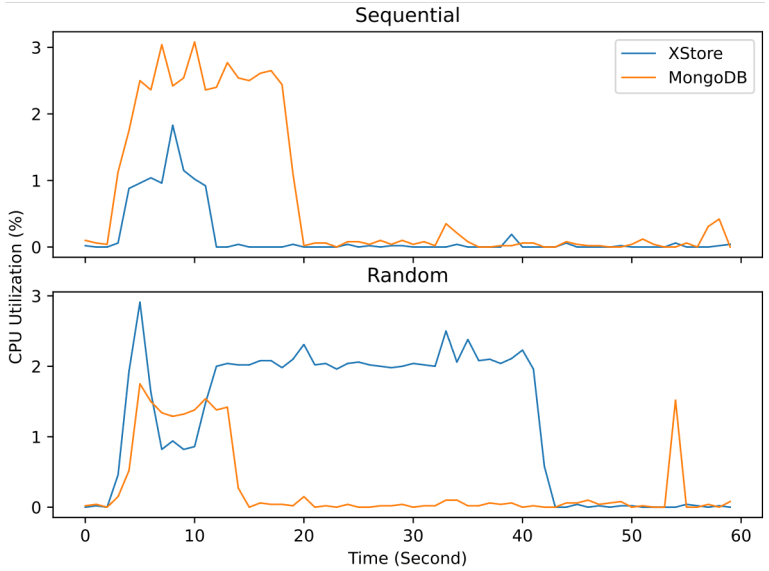
Benchmark – Unary Insert



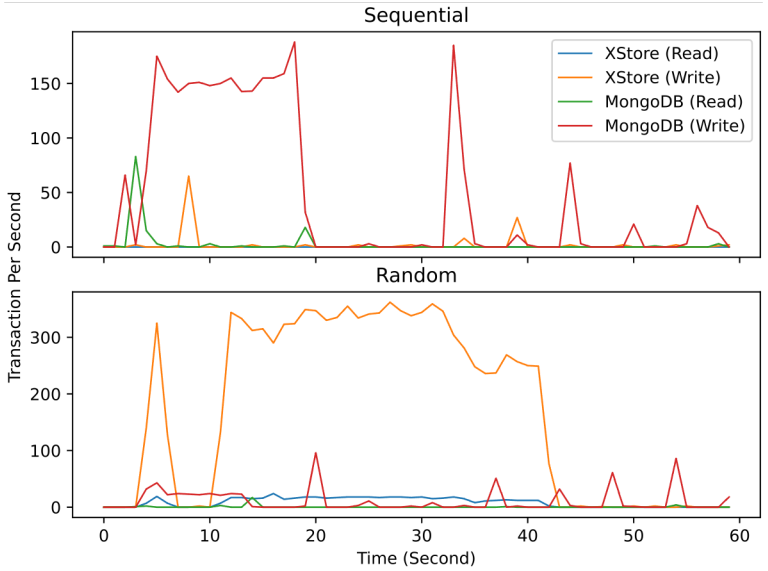
	Min	Max	Mean	Std Dev	Avg. Throughput
XStore (SEQ)	0.55	0.87	0.74	0.04	1,352.65
XStore (RAND)	0.59	0.93	0.76	0.05	1,307.44
XStore (Ram Disk)	0.55	0.96	0.75	0.05	1,336.26
MongoDB (SEQ)	0.78	1.95	1.34	0.24	744.98
MongoDB (RAND)	0.77	1.10	0.93	0.06	1,070.60
MongoDB (2nd Index)	0.86	19.06	1.45	0.86	688.02
MongoDB (Ram Disk)	0.85	31.27	1.41	1.03	711.08

Note: Replaced 25 observations for MongoDB with latency ≥ 2 (ms) with its respective average
Ram Disk benchmarks are run with Random Workload

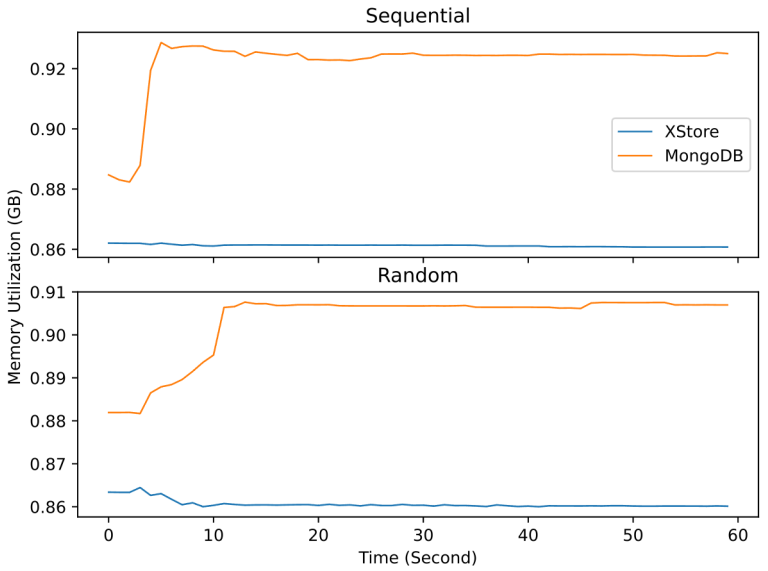
Benchmark – Unary Insert (System Activity)



CPU



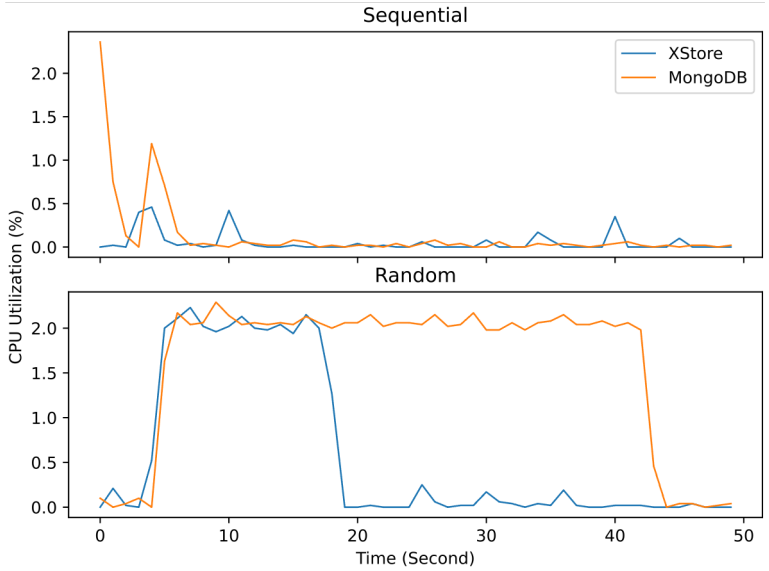
I/O



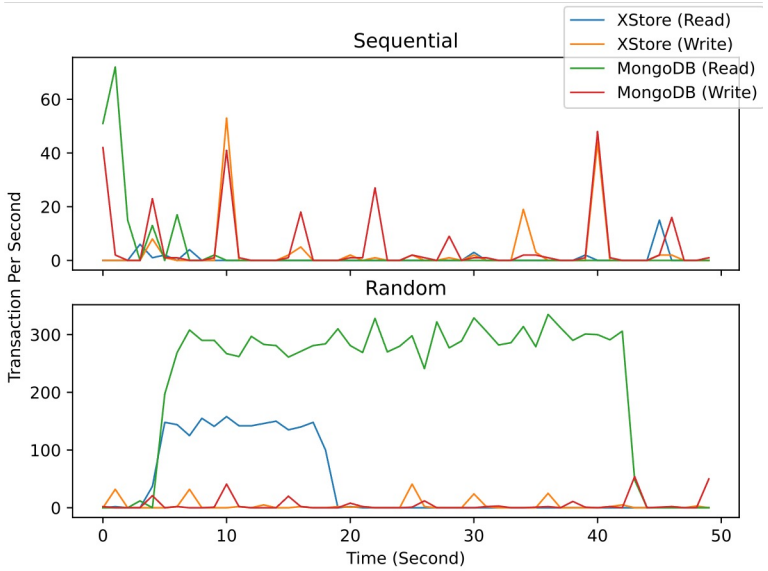
MEM

Note: Ran 10,000 unary inserts to adequately capture the system resource activity

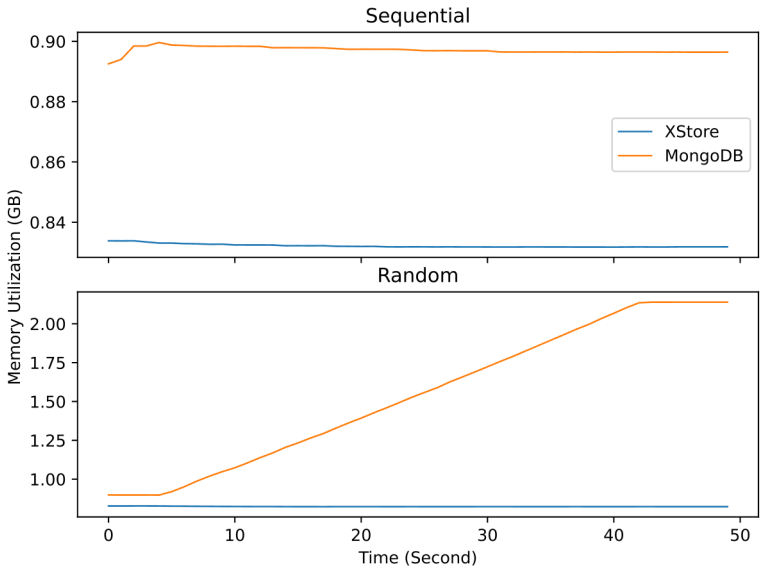
Benchmark – Unary Query (System Activity)



CPU



I/O



MEM

Note: Ran 1,000 unary queries to adequately capture the system resource activity

MongoDB Compression*

1 Network

As a distributed database, MongoDB relies on efficient network transport during query routing and inter-node replication. Based on the snappy compression algorithm, network traffic across a MongoDB cluster can be compressed by up to 80%, providing major performance benefits in bandwidth-constrained environments, and reducing networking costs.

2 Storage

The default snappy compression reduces storage footprint typically by 50% or more, and enables higher IOPs as fewer bits are read from disk. As with any compression algorithm, you trade storage efficiency for CPU overhead, and so it is important to test the impacts of compression in your own environment.

MongoDB offers you a range of compression options for both documents and indexes. The snappy compression algorithm provides a balance between high document compression ratios (typically around 50%+, dependent on data types) with low CPU overhead, while the optional zStandard and zlib libraries will achieve higher compression, but incur additional CPU cycles as data is written to and read from disk. zStandard was introduced with the MongoDB 4.2 release and is recommended over the existing zLib library due to lower CPU overhead.

*https://www.mongodb.com/blog/post/performance-best-practices-hardware-and-os-configuration